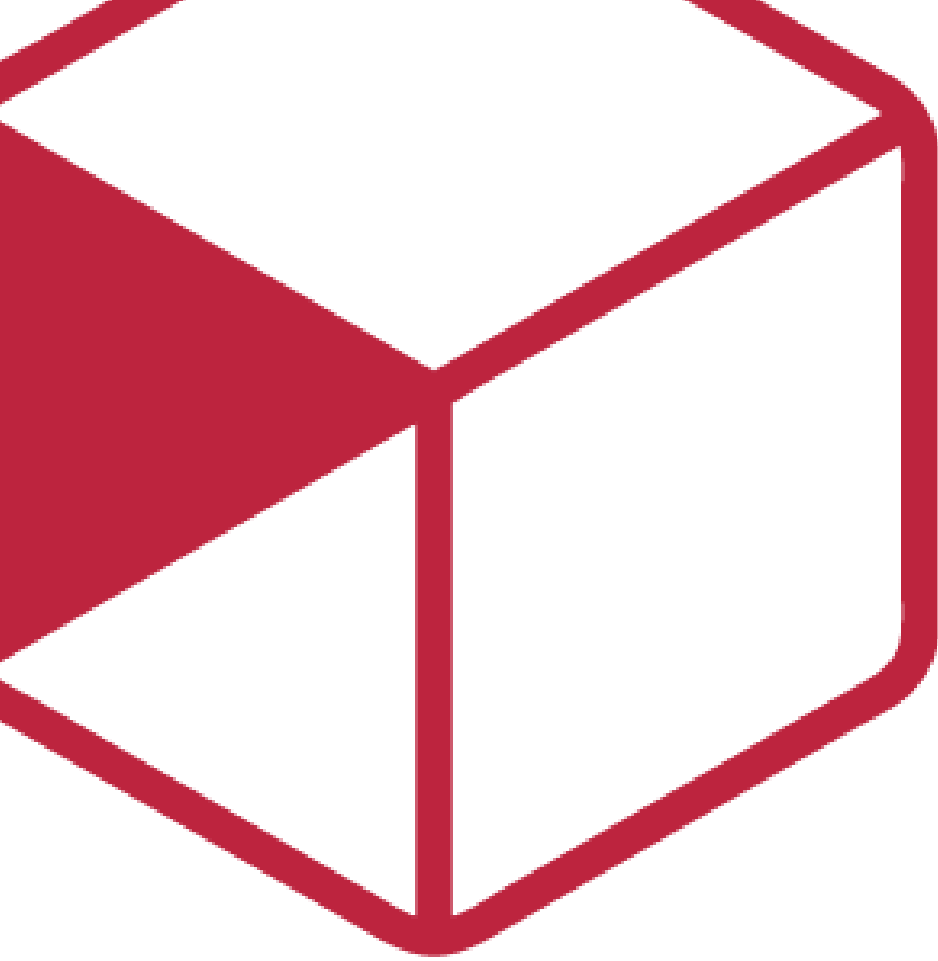




Diseño de la solución

Requisito R16A-RE-FU-026

FORMATO	Arquitectura
PROYECTO	R16 - Adquisiciones
REFERENCIA	AUI- FOR-01
VERSIÓN	1.0
FECHA	01/07/2026
AUTOR	Isai Amaury Garcia Flores
REVISOR	Valdemar Farina Sanchez

Contenido

Importante	3
Introducción	4
1. Propósito del documento	4
2. Alcance	4
Específicamente incluye:	4
No se consideran:	5
**En caso de aplicar colocar lista de consideraciones que no aplicaran	5
Visión general del diseño	6
1. Objetivo técnico	6
2. Componentes involucrados	6
Diseño funcional detallado	7
1. Carga inicial y filtrado del historial	7
2. Criterios de aceptación del requisito	8
3. Reglas técnicas aplicadas	9
Diseño de componentes	10
1. Diagramas	10
Impacto Técnico	11
1. Impacto en código existente	11
2. Impacto en modelos por actualización de modelos	11
Manejo de Errores y Excepciones	12
Estrategia de Pruebas (Diseño de las pruebas)	12
1. Pruebas funcionales (Criterios de Aceptación en DEV)	13
2. Pruebas técnicas(unitarias e integración)	13
a. Unitarias	13
b. Pruebas de integración	13
3. Casos críticos	13
Control de versiones	14



Importante

Posterior a este diseño, ¿cómo saber si el diseño de la solución al requisito está completo para que el programador inicie con el desarrollo?

Hazte estas preguntas rápidas:

- ¿El programador sabe qué flujo implementar?
- ¿Sabe qué pasa si algo falla?
- ¿Sabe qué reglas no puede romper?
- ¿Sabe qué pruebas debe pasar?
- ¿Sabe dónde impacta?

Nota: Este documento es una propuesta basada en el estándar IEEE 1016 “Software Design Description” que va permitir abarcar los puntos más importantes para el diseño del requisito que se está trabajando. Se debe administrar el tiempo que se tiene de diseño para que se complete de la mejor manera considerando todos los detalles técnicos.



Introducción

1. Propósito del documento

Definir el diseño técnico backend para el requerimiento R16A-RE-FU-026 (Paso 2 - Asociación del wizard Validar Cobro, Región México), describiendo:

- Arquitectura de servicios y APIs
- Modelo de datos (tablas nuevas y alteraciones)
- Flujos técnicos transaccionales
- Contratos REST entre Frontend (Angular) y Backend (ProquifaDotNet.Finanzas)
- Reglas de implementación
- Impacto en código y base de datos existentes

2. Alcance

Este análisis cubre únicamente:

- Alteraciones de BD en ProquifaDotNet (.NET Framework 4.8)
- Servicios REST en ProquifaDotNet.Finanzas (.NET Core 10)
- Lógica de orquestación del Paso 2 (asociación N:N, cálculo saldo, escenarios)
- Cálculo multi-divisa con TC capturado en Paso 1
- Persistencia transaccional de asociaciones
- Señalización de escenario insuficiente (bloqueo de avance) para que Frontend active el modal de inconsistencia
- Auto-guardado del estado en progreso
- Integración con módulo Notas de Crédito (lectura)
- Integración con módulo Validar Cobro Paso 1 (lectura de cobros y TC)

Específicamente incluye:

- Creación tabla fccSaldoFavorCliente con ClaveMoneda varchar(3) + ClaveRegion varchar(3) (region-ready — evita ALTER para Perú y monedas futuras)
- Lectura de Notas de Crédito vigentes del cliente (tabla y estructura por definir en diseño R032; este requisito consume los datos pero no define ni crea la tabla)
- 5 servicios REST en Finanzas: listado docs pendientes, catálogo NCs, persistencia asociaciones, cálculo saldo dinámico, auto-guardado
- Strategy pattern (IPaso2RegionStrategy + Paso2StrategyBase + Paso2MxStrategy + ResolverStrategy) — solo MX implementada en R026
- Tolerancia configurable vía appsettings.json (clave Finanzas:ToleranciaValidarCobroMXN = 100)
- Transacción ACID multi-tabla con rollback + gate fiscal post-COMMIT (RequiereEfectoFiscal)



- Validación escenarios region-neutral (pago exacto, sobrepago, tolerancia $\leq$ umbral, insuficiente $>$ umbral)
- Conversión de moneda por TC Paso 1
- DTOs region-aware (Region, MonedaBase, UmbralTolerancia, EfectoFiscal) con valores MX por defecto

No se consideran:

- Implementación del Frontend (Angular/NgRx) — se documenta en diseño frontend paralelo
- Diseño de Notas de Crédito (módulo independiente — R032, a cargo de otro desarrollador); la tabla, columnas y contratos de ese módulo se definen allí, no aquí
- Diseño del Paso 1 (Captura Cobro) — ya existe R16A-RE-FU-024
- Diseño del Paso 3 (Facturación Envío) — pendiente R16A-RE-FU-025
- Modal de Inconsistencia y registro de inconsistencias del Paso 2 — contemplado en R16A-RE-FU-024 (diseño aún no realizado; endpoint y lógica de `fccInconsistenciaCobro` / `catTipoInconsistenciaCobro` se definirán allí)
- Estilos, validaciones de frontend o reglas UX



Visión general del diseño

1. Objetivo técnico

Implementar en **ProquifaDotNet.Finanzas** la orquestación completa del Paso 2 del wizard Validar Cobro, permitiendo que el usuario:

- Cargue cobros capturados + proformas/facturas pendientes + Notas de Crédito vigentes
- Asocie cobros con documentos en relación N:N (prioridad por orden de selección)
- Aplique opcionalmente Notas de Crédito por documento
- Visualice cálculo dinámico de saldo en moneda del cobro (conversión por TC)
- Resuelva 4 escenarios: pago exacto (avanza), sobrepago (registra saldo a favor), tolerancia ≤ 100 MXN (avanza + saldo pendiente), insuficiente > 100 MXN (bloquea y señala al Frontend para activar modal de inconsistencia — implementado en R16A-RE-FU-024)
- Marque inconsistencias contextuales del Paso 2 (Pago Incompleto Vencido, Pago Insuficiente)
- Persista asociaciones transaccionalmente con rollback garantizado

Todo con auto-guardado transparente y editabilidad mientras permanece en el Paso 2.

2. Componentes involucrados

Identificar **componentes reales** del sistema que participan e indicar la responsabilidad de cada uno.

Aplicativo	Componente	Responsabilidad	Ubicación
ProquifaDotNet	SQL Server	Almacenar tablas de asociación, saldos, tipos inconsistencia	Base de datos ProquifaDotNet
ProquifaDotNet.Finanzas	RestController (ASP.NET Core)	Exponer endpoints Paso 2	Controllers/Payment Validation/Step2Controller.cs
ProquifaDotNet.Finanzas	Services (orquestación)	Lógica multi-divisa, cálculo saldo, escenarios, transacciones	Services/PaymentValidation/Step2Service.cs
ProquifaDotNet.Finanzas	Entity Framework / DAL	Acceso y manipulación de datos	Data/Repositories/Pass2Repository.cs

Diseño funcional detallado

Descripción paso a paso del flujo técnico con orden lógico (**si alguien más lo lee, pueda dibujarlo en un diagrama de flujo**). Se ponen tantos flujos como se hayan identificado.

Por ejemplo:

1. Flujo 1: Carga Inicial del Paso 2

Trigger: Usuario avanza desde Paso 1 a Paso 2 con al menos 1 cobro capturado.

Secuencia backend:

- Frontend invoca: `/api/v1/validate-collection/client/{idCliente}/pendingDocument`
- Backend (Paso2Service) obtiene:
 - Proformas normales pendientes: `SELECT tpProformaPedido WHERE IdCliente=@id AND MontoPendiente>0 AND Cancelada=0 AND Activo=1`
 - Facturas por Adelantado pendientes: `SELECT tpProformaAdelanto WHERE IdCliente=@id AND MontoPendiente>0 AND Cancelada=0 AND Activo=1`
 - Notas de Crédito vigentes: pendiente R032 — la tabla, columnas y query exacta se definen en el diseño de ese requisito. Este servicio debe retornar el listado una vez R032 exponga el contrato de lectura.
 - Estado del Paso 2 previo (si existe): `SELECT * FROM fccEstadoPaso2EnProgreso WHERE IdCliente=@id AND IdSesionValidarCobro=@sesion`
- Backend enriquece cada documento:
 - Agrega empresa emisora (Alias: GOL, MUN, PRO, PQF)
 - Agrega moneda (ClaveMoneda: MXN, USD, etc.)
- Retorna DTO (Json) con estructura:

```
{
  "clientId": "uuid",
  "pendingDocuments": [
    {
      "documentId": "uuid",
      "type": "PROFORMA",
      "folio": "PRF-062926-001",
      "internalOrderNumber": "PED-12345",
      "emitterCompany": "PRO",
      "totalAmount": 10000.00,
    }
  ]
}
```



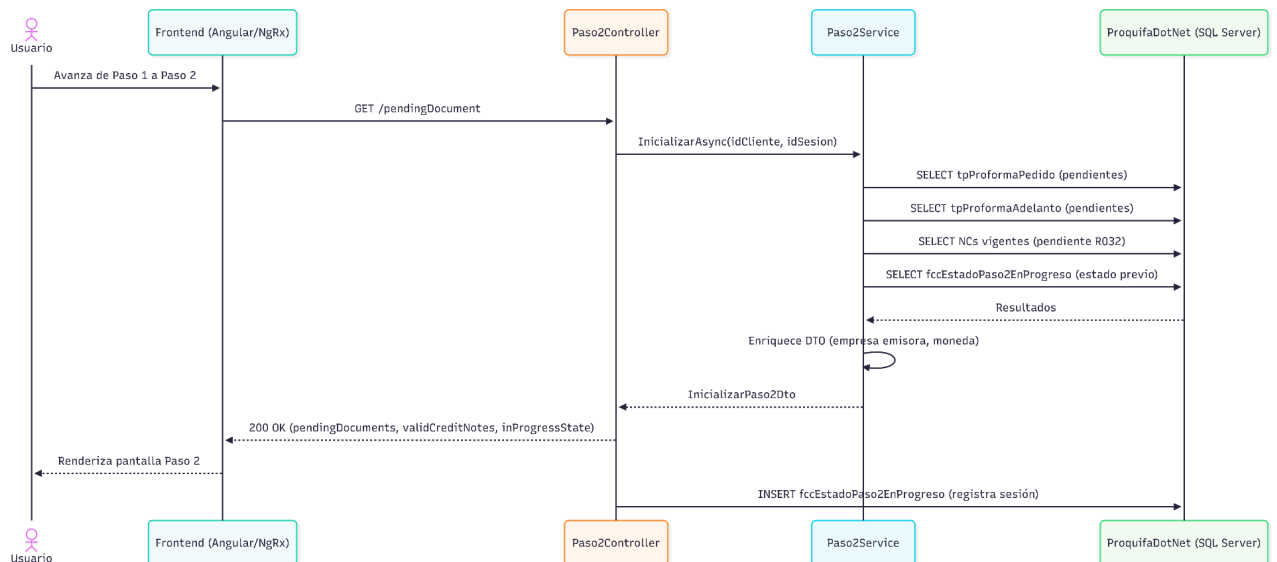
```

    "pendingBalance": 10000.00,
    "originalCurrency": "MXN",
    "documentType": "tpProformaPedido"
  }
],
"validCreditNotes": [
  {
    // Structure pending -- fields and types defined in R032.
    // This contract will be updated once R032 establishes the credit
    note model.
  }
],
"inProgressState": {
  "selectedPayments": [],
  "selectedDocuments": [],
  "appliedCreditNotes": {},
  "inconsistency": null
}
}

```

- Frontend renderiza pantalla Paso 2 con datos iniciales
- Backend registra sesión Paso 2 para auto-guardado: INSERT fccEstadoPaso2EnProgreso (...)

Diagrama de secuencia:



2. Flujo 2: Cálculo Dinámico de Saldo

Trigger: Frontend selecciona/deselecciona cobros, documentos o aplica/remueve NCs.

Proceso:

- Frontend invoca: POST /api/v1/validate-collection/client/{idCliente}/balance/calculate con payload:

```
{
  "clientId": "uuid",
  "selectedPayments": ["uuid-payment-1", "uuid-payment-2"],
  "selectedDocuments": ["uuid-doc-1", "uuid-doc-3"],
  "appliedCreditNotes": {
    "uuid-doc-1": ["uuid-creditnote-1", "uuid-creditnote-2"],
    "uuid-doc-3": ["uuid-creditnote-3"]
  }
}
```

- Backend (Paso2Service.CalcularSaldo) ejecuta:
 - Suma cobros aplicados:
 - Por cada cobro en cobrosSeleccionados: traer monto desde fccPagoCliente
 - Convertir a moneda del primer cobro (TC desde fccPagoCliente.TipoCambio)
 - Suma total = SumaCobros
 - Suma NCs Aplicadas:
 - Por cada NC en ncsAplicadas: traer monto desde fccNotaCredito
 - Convertir a moneda del cobro usando TC (si moneda NC ≠ moneda cobro)
 - Suma total = SumaNCs
 - Suma Adeudo Documentos:
 - Por cada documento en documentosSeleccionados: traer MontoPendiente
 - Convertir a moneda del cobro usando TC del Paso 1:
 - Si documento en MXN, cobro en USD: dividir MontoPendiente ÷ TC
 - Si documento en USD, cobro en MXN: multiplicar MontoPendiente × TC
 - Suma total = SumaAdeudo
 - Cálculo Saldo: $\text{SaldoAsociacion} = (\text{SumaCobros} + \text{SumaNCs}) - \text{SumaAdeudo}$



- Determinación escenario (delegada a strategy.DeterminarEscenario(saldo) — umbral leído de config, no hardcodeado)::

```
UmbralTolerancia = strategy.ObtenerUmbralTolerancia() // MX: config
'Finanzas:ToleranciaValidarCobroMXN' (100 por defecto)
IF SaldoAsociacion == 0.00
    Escenario = 'PAGO_EXACTO' → Permite avance
ELSE IF SaldoAsociacion > 0
    Escenario = 'SALDO_A_FAVOR' → Permite avance, registra saldo a favor
// alineado a catCobroEstatus.SALDO_A_FAVOR (propuesta revisor)
ELSE IF SaldoAsociacion < 0 AND ABS(SaldoAsociacion) <= UmbralTolerancia
    Escenario = 'TOLERANCIA' → Permite avance, registra tolerancia //
nombre region-neutral
ELSE
    Escenario = 'PAGO_INSUFICIENTE' → Bloquea avance, requiere
inconsistencia
```

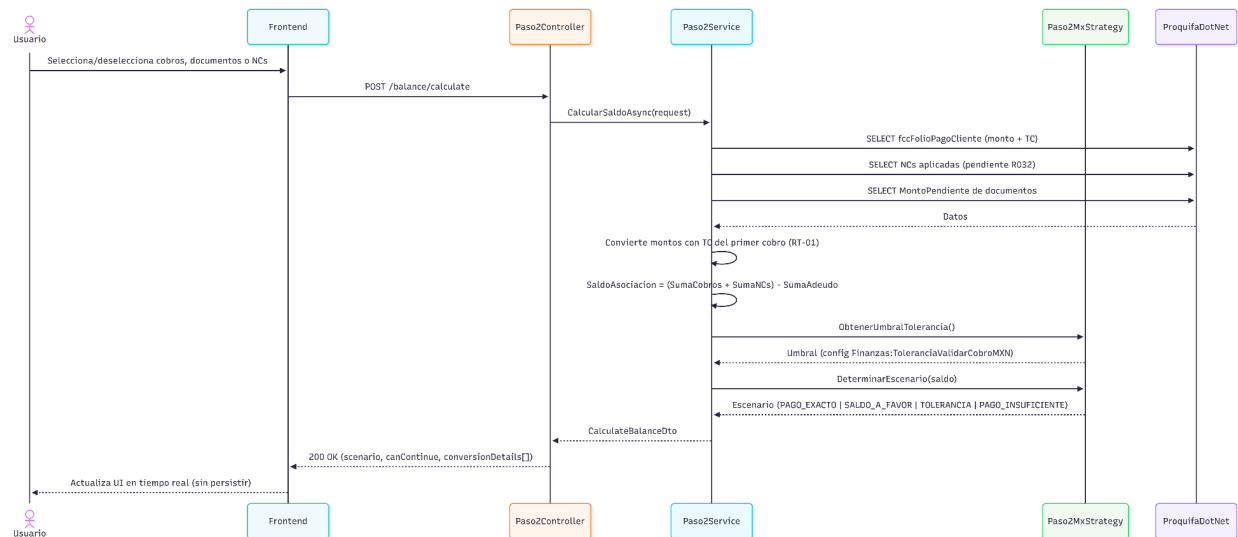
- Backend retorna:

```
{
  "paymentSum": 5000.00,
  "creditNoteSum": 500.00,
  "debtSum": 4500.00,
  "associationBalance": 1000.00,
  "scenario": "SALDO_A_FAVOR",
  "canContinue": true,
  "finalCurrency": "MXN",
  "appliedToleranceThreshold": 100.00,
  "conversionDetails": [
    {
      "documentId": "uuid-doc-3",
      "originalCurrency": "USD",
      "originalAmount": 4500.00,
      "exchangeRate": 1.00,
      "convertedAmount": 4500.00,
      "formula": "4500.00 × 1.00 = 4500.00"
    }
  ]
}
```

- Frontend recibe respuesta y actualiza UI en tiempo real (sin persistir aún)



Diagrama de secuencia:



3. Flujo 3: Persistencia de Asociaciones (Transacción ACID)

Trigger: Usuario presiona "Continuar" y saldo es exacto, sobrepago, o tolerancia ≤ 100 MXN.

Secuencia transaccional:

- Frontend invoca: POST `/api/v1/validate-collection/client/{idCliente}/associationConfirmation` con payload final:

```

{
  "clientId": "uuid",
  "idSessionId": "uuid",
  "selectedPayments": ["uuid-payment-1"],
  "selectedDocuments": ["uuid-doc-1", "uuid-doc-3"],
  "appliedCreditNotes": {
    "uuid-doc-1": ["uuid-creditnote-1"],
    "uuid-doc-3": ["uuid-creditnote-3"]
  },
  "scenario": "OVERPAYMENT",
  "associationBalance": 1000.00
}

```

- Backend abre transacción (ACID, isolation level Serializable):

Paso A1: Validar integridad (cobros/docs/NCs aún existen, montos coherentes)

Paso A2: INSERT fccPagoFacturaPedido (por cada cobro ↔ proforma normal)

```
INSERT INTO dbo.fccPagoFacturaPedido (  
    IdFCCPagoFacturaPedido, IdFCCFolioPagoCliente, IdTPProformaPedido,  
    MontoAplicado, TipoCambio, FechaRegistro, FechaUltimaActualizacion,  
    Activo  
)  
VALUES (  
    NEWID(), @idCobro, @idProforma,  
    @montoAplicado, @tipoCambio, SYSUTCDATETIME(), SYSUTCDATETIME(), 1  
)
```

Paso A3: INSERT fccPagoFacturaAdelanto (por cada cobro ↔ FAA)

```
INSERT INTO dbo.fccPagoFacturaAdelanto (  
    IdFCCPagoFacturaAdelanto, IdFCCFolioPagoCliente, IdTPProformaAdelanto,  
    MontoAplicado, TipoCambio, FechaRegistro, FechaUltimaActualizacion,  
    Activo  
)  
VALUES (...)
```

Paso A4: Marcar NCs como aplicadas — pendiente R032

La operación de marcar una NC como aplicada (columna, lógica y tabla exacta) se define en el diseño de R032. Este paso debe coordinarse con el desarrollador de ese requisito antes de implementar R026. Por ahora solo se documenta la intención: las NCs seleccionadas en esta sesión quedan vinculadas al cobro y no deben estar disponibles para sesiones futuras.

Paso A5: INSERT fccSaldoFavorCliente (si sobrepago o tolerancia) — usa ClaveRegion y ClaveMoneda (region-neutral, sin bits por moneda):

```
IF @escenario IN ('SALDO_A_FAVOR', 'TOLERANCIA')  
    INSERT INTO dbo.fccSaldoFavorCliente (  
        IdFCCSaldoFavorCliente, IdCliente, IdFCCFolioPagoCliente,  
        ClaveRegion, TipoSaldo, Monto, ClaveMoneda, TipoCambio,  
        Aplicado, Observaciones, Activo, FechaRegistro  
    )  
    VALUES (  
        NEWID(), @idCliente, @idCobro,  
        @claveRegion, -- 'MEX' para MX (provisto por strategy.ClaveRegion)  
        CASE WHEN @escenario = 'SALDO_A_FAVOR' THEN 'SaldoFavor' ELSE
```



```
'ToleranciaAplicada' END,  -- TipoSaldo sigue PascalCase (fuera de alcance
de este cambio; ver v1.3)
    ABS(@saldoAsociacion),
    @claveMoneda,  -- 'MXN' para MX (provisto por strategy.MonedaBase o
moneda del cobro)
    @tipoCambio, 0, @observaciones, 1, SYSUTCDATETIME()
)
```

Paso A6: UPDATE fccPagoCliente (marcar cobro con "saldo a favor" si aplica)

```
IF @escenario IN ('SALDO_A_FAVOR', 'TOLERANCIA')
    UPDATE dbo.fccFolioPagoCliente
    SET TieneSaldoAFavor = 1, FechaUltimaActualizacion = SYSUTCDATETIME()
    WHERE IdFCCFolioPagoCliente = @idCobro
```

Paso A7: DELETE fccEstadoPaso2EnProgreso (limpiar estado temporal)

```
DELETE FROM dbo.fccEstadoPaso2EnProgreso
WHERE IdCliente = @idCliente AND IdSesionValidarCobro = @idSesion
```

Paso A8: COMMIT transacción (éxito) O ROLLBACK (cualquier error)

Post-COMMIT — Gate fiscal (fuera de la transacción ACID, solo si COMMIT exitoso):

```
// Solo regiones con efecto fiscal (MX: RequiereEfectoFiscal = true).
// PE (futuro): RequiereEfectoFiscal = false → no entra al bloque.
if (strategy.RequiereEfectoFiscal)
{
    await _fiscalQueue.EnqueueComplementoPagoAsync(idAsociacion);
}
```

- Backend retorna:

```
{
  "success": true,
  "associationId": "uuid-session",
  "message": "Association confirmed. Advancing to Step 3.",
  "scenario": "SALDO_A_FAVOR",
  "registeredBalance": 1000.00,
  "fiscalEffect": true,
  "sunatNote": null
}
```



```
-- DML inicial
INSERT INTO dbo.catCobroEstatus (Clave, Descripcion, Orden) VALUES
  ('BORRADOR', 'Captura iniciada en Paso 1, no confirmada', 1),
  ('CAPTURADO', 'Cobro confirmado en Paso 1, pendiente de asociar', 2),
  ('ASOCIADO', 'Vinculado a proforma o factura en Paso 2', 3),
  ('SALDO_A_FAVOR', 'Cobro con residual disponible tras asociación', 4),
  ('CON_INCONSISTENCIA', 'Marcado con inconsistencia en Paso 1 o Paso 2', 5),
  ('COMPLETADO', 'Documentos fiscales generados y enviados en Paso 3', 6),
  ('CANCELADO', 'Cancelado por falta de pago u otra razón operativa', 7);

```

Ciclo de vida

[Correo clasificado como Cobro → INSERT fccFolioPagoCliente]

```

  ↓
BORRADOR      ← Auto-guardado Paso 1 (Confirmado=0)
  ↓ (confirma cobro)
CAPTURADO      ← Paso 1 completo (Confirmado=1, folio COB-mmddaa-NNNN)
  ↓ (asocia a documento en Paso 2)
ASOCIADO o SALDO_A_FAVOR
  ↓ (Paso 3 genera y envía documento fiscal)
COMPLETADO

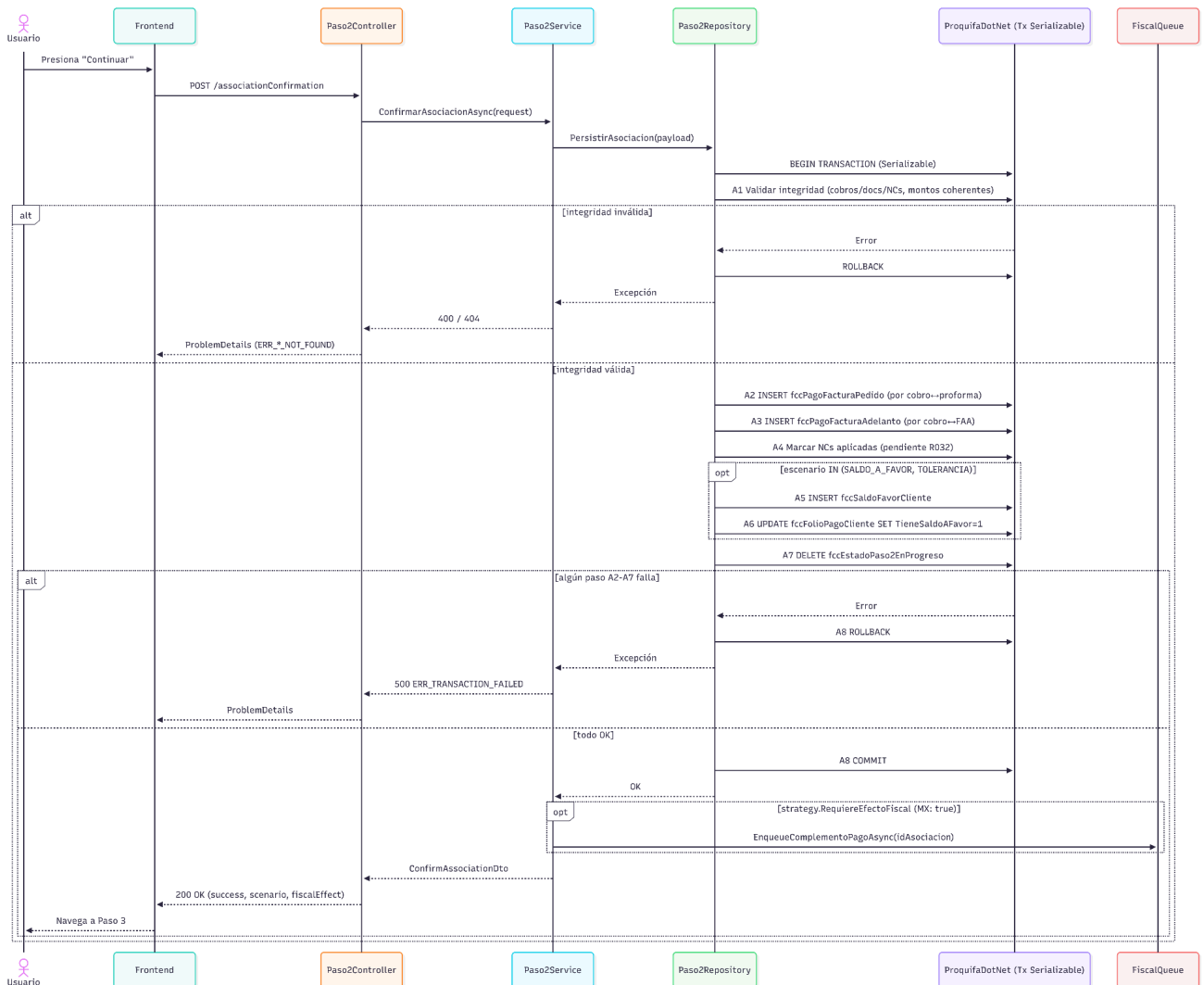
```

En cualquier punto → CON_INCONSISTENCIA o CANCELADO

- Front navega al paso 3



Diagrama de secuencia:



Si falla cualquier paso: rollback completo de toda transacción, estado BD consistente.

4. Flujo 4: Modal de Inconsistencia del Paso 2

Este flujo no se implementa en este requisito. El Modal de Inconsistencia (registro en `fccInconsistenciaCobro`, catálogo `catTipoInconsistenciaCobro`, opción de marcar pedido para cancelación) está contemplado en R16A-RE-FU-024, front consume el endpoint con filtro según el paso que lo consulta.

5. Auto-Guardado del Paso 2

Trigger: Cada 10 segundos o al cambiar selección de cobros/docs/NCs mientras usuario esté en Paso 2.

Mecanismo:

- Frontend invoca periódicamente (cada 10 segundos): *POST* */api/v1/validate-collection/client/{idCliente}/association/draft* con estado actual:

```
{
  "clientId": "uuid",
  "idSessionId": "uuid",
  "selectedPayments": ["uuid-payment-1"],
  "selectedDocuments": ["uuid-doc-1"],
  "appliedCreditNotes": {"uuid-doc-1": ["uuid-creditnote-1"]},
  "inconsistency": null,
  "timestamp": 1719701400
}
```

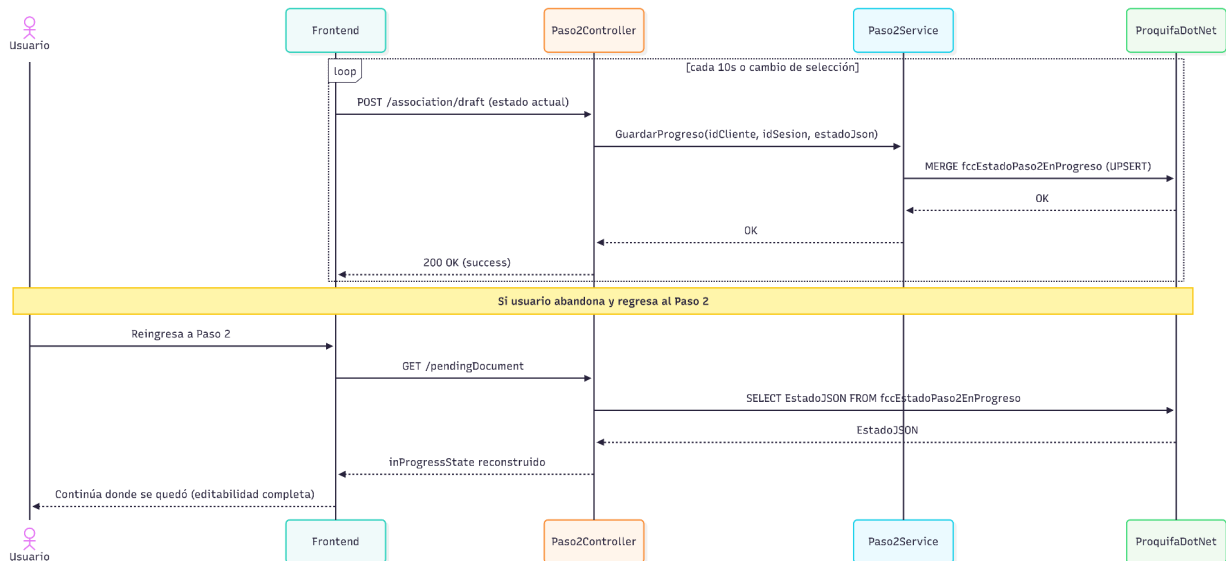
- Backend upsert en tabla fccEstadoPaso2EnProgreso:

```
MERGE INTO dbo.fccEstadoPaso2EnProgreso AS target
USING (SELECT @idCliente, @idSesion, @estado) AS source (IdCliente,
IdSesion, Estado)
ON (target.IdCliente = source.IdCliente AND target.IdSesion =
source.IdSesion)
WHEN MATCHED THEN
  UPDATE SET EstadoJSON = @estado, FechaUltimaActualizacion =
SYSUTCDATETIME()
WHEN NOT MATCHED THEN
  INSERT (IdFCCEstadoPaso2, IdCliente, IdSesion, EstadoJSON, FechaRegistro)
VALUES (NEWID(), @idCliente, @idSesion, @estado, SYSUTCDATETIME());
```

- Si usuario abandona y regresa al paso 2:
 - Backend carga desde fccEstadoPaso2EnProgreso: SELECT EstadoJSON
 - Reconstruye estado y lo entrega a Frontend
 - Usuario continúa donde se quedó (editabilidad completa)
- Una vez avanzada a paso 2: DELETE fccEstadoPaso2EnProgreso (estado temporal se descarta)



Diagrama de secuencia:



6. Flujo 6: Navegación (Regresar vs Continuar)

Botón regresar:

- Frontend: POST /api/v1/validate-collection/client/{idCliente}/association/draft (preserva estado)
- Navega a Paso 1
- Paso 2 queda guardado automáticamente

Botón continuar:

- Valida escenario (debe ser exacto, sobrepago, o tolerancia ≤ 100 MXN)
- Si válido: ejecuta Flujo 3 (Persistencia de Asociaciones)
- Navega a Paso 3
- Paso 2 se archiva (fccEstadoPaso2EnProgreso se elimina)

7. Criterios de aceptación del requisito

Realizar un mapeo explícito de los criterios de aceptación (CA) que demuestre que TODOS están cubiertos.

Ejemplo:

CA	Descripción	Estado	Justificación
A1	Dado que el usuario entra al Paso 2 del wizard, Cuando el sistema muestra la cabecera del cliente, Entonces deberá mostrar: logo del cliente (si existe), Alias, etiquetas preexistentes de clasificación del Catálogo de Clientes, identificador fiscal bajo la etiqueta unificada "RFC/RUC", razón social legal completa y moneda de facturación.	Cubierto ▾	Frontend renderiza con datos de GET /inicializar; backend proporciona logo, Alias, etiquetas, RFC/RUC, razón social, moneda facturación
A2	Dado que el usuario está en el Paso 2, Cuando el sistema muestra la barra de pasos, Entonces deberá mostrar los tres pasos del wizard: "1 - CAPTURAR COBRO" (✓), "2 - ASOCIAR FACTURA/PROFORMA" (activo), "3 - FACTURACIÓN Y ENVÍO".	Cubierto ▾	Frontend renderiza barra; backend valida que usuario está en Paso 2 y retorna estado sesión
B1	Dado que el cliente tiene cobros capturados confirmados en el Paso 1,	Cubierto ▾	Frontend reutiliza cobros del Paso 1 (NgRx state). Backend no los re-carga; incluye flag



	Cuando el sistema muestra el listado de cobros disponibles para asociar, Entonces deberá mostrar cada cobro con: checkbox de selección, folio definitivo (COB-mmddaa-NNNN), Fecha del Cobro, etiqueta del monto ("Monto del cobro" cuando el cobro no arrastra residual; "Saldo a favor" cuando arrastra residual de sesión anterior) con su moneda, e icono de correo (✉) que abre pop-up con los datos del correo origen del cobro. El listado se ordena por Fecha del Cobro, de la más antigua a la más reciente, consistente con el Paso 1.		tieneSaldoAFavor en DTO
B2	Dado un item de cobro en el listado, Cuando el usuario presiona el icono de correo (✉), Entonces el sistema deberá abrir un pop-up modal con los datos del correo origen del cobro: asunto, fecha y hora de llegada, cuerpo del correo y el comprobante de pago identificado por el usuario en el Paso 1 (marcado como activo). El pop-up es de solo lectura, sirve como referencia contextual para validar el cobro sin alterar la asociación en curso, y se cierra mediante acción explícita del usuario regresando al Paso 2. Los datos del cobro no se editan desde este pop-up (inmutabilidad confirmada en el Paso 1).	Cubierto ▾	Backend retorna flag en fccPagoClient e.TieneSaldoAFavor; Frontend renderiza etiqueta visual
B3	Dado que el listado de cobros está visible,	Cubierto ▾	Frontend maneja checkboxes; backend



	Cuando el usuario selecciona uno o más cobros mediante checkboxes, Entonces el sistema deberá habilitar la asociación de esos cobros con uno o más documentos pendientes de cobrar.		acepta array cobrosSeleccionados[] en POST /calcular-saldo
B4	Dado un item de cobro en el listado, Cuando el usuario presiona el icono de correo (✉), Entonces el sistema deberá abrir un pop-up modal con los datos del correo origen del cobro: asunto, fecha y hora de llegada, cuerpo del correo y el comprobante de pago identificado por el usuario en el Paso 1 (marcado como activo). El pop-up es de solo lectura, sirve como referencia contextual para validar el cobro sin alterar la asociación en curso, y se cierra mediante acción explícita del usuario regresando al Paso 2. Los datos del cobro no se editan desde este pop-up (inmutabilidad confirmada en el Paso 1).	Cubierto ▾	Frontend renderiza pop-up; backend incluye referencia correo en DTO de GET /inicializar
B5	Dado que el cliente NO tiene cobros capturados confirmados en el Paso 1, Cuando el usuario accede al Paso 2, Entonces el sistema deberá mostrar un mensaje indicativo: "No hay cobros disponibles. Regrese al Paso 1 para capturar al menos un cobro" y deshabilitar el acceso a las siguientes secciones (C, D, E, F, H) hasta que haya al menos un cobro seleccionado.	Cubierto ▾	Frontend valida que al menos 1 cobro esté seleccionado antes de invocar Flujo 2; backend retorna error 400 si se intenta sin cobros
C1	Dado que el cliente tiene proformas y/o facturas pendientes de cobrar,	Cubierto ▾	GET /inicializar retorna proformas +



	Cuando el sistema muestra el listado central, Entonces deberá mostrar uno por uno los documentos con: tipo (Proforma o Factura), folio del documento, Pedido Interno, empresa emisora del grupo PROQUIFA México (Golocaer, Mungen, Proquifa, Proveedora Quimico Farmaceutica), importe total, monto a aplicar y saldo pendiente actual.		FAAs con tipo, folio, Pedido Interno, empresa emisora, importe Total, saldo Pendiente, moneda
C2	Dado que el usuario opera el listado de documentos, Cuando consulta los filtros disponibles, Entonces el sistema deberá ofrecer el listado completo sin filtros adicionales por tipo, empresa emisora, fecha u otros criterios. El usuario selecciona libremente los documentos a asociar.	Cubierto ▾	Backend retorna listado sin aplicar filtros; Frontend puede ofrecer UI para seleccionar sin alterar datos servidor
C3	Dado que el cliente tiene documentos pendientes emitidos por diferentes empresas del grupo PROQUIFA México, Cuando el sistema muestra el listado, Entonces deberá mostrar todos los documentos pendientes mezclados (de diferentes empresas emisoras) en el mismo listado. La empresa emisora de cada documento se conserva para procesamiento independiente en el Paso 3.	Cubierto ▾	Backend retorna documentos de todas empresas (GOL, MUN, PRO, PQF); cada uno conserva su empresa Emisora para Paso 3
C4	Dado que el cliente NO tiene proformas ni facturas pendientes de cobrar, Cuando el usuario accede al Paso 2 con al menos un cobro seleccionado,	Cubierto ▾	Frontend valida que al menos 1 doc esté seleccionado; backend retorna error



	Entonces el sistema deberá mostrar un mensaje indicativo: "No hay documentos pendientes para este cliente. Verifique en Gestionar Cobranza o Seguimiento de Pedidos" y deshabilitar el avance a las secciones D, E, F, H hasta que haya al menos un documento pendiente disponible.		400 si lista está vacía
C5	Dado que el listado de documentos está visible, Cuando el usuario consulta el orden de presentación, Entonces el sistema deberá mostrar los documentos ordenados por fecha de vencimiento (vencidos primero, luego próximos a vencer). Este orden facilita que el usuario priorice los documentos vencidos.	Cubierto ▾	GET /inicializar ordena docs por fecha vencimiento (vencidos primero); Frontend mantiene orden
D1	Dado que el usuario seleccionó uno o más cobros y uno o más documentos, Cuando arma la asociación, Entonces el sistema deberá permitir las combinaciones: $1 \leftrightarrow 1$, $1 \leftrightarrow N$, $N \leftrightarrow 1$, $N \leftrightarrow N$. El usuario decide manualmente cuáles cobros se aplican a cuáles documentos sin asistencia automática del sistema.	Cubierto ▾	Frontend maneja checkboxes; backend acepta combinaciones N:N en POST /calcular-saldo
D2	Dado que el usuario asocia cobros con documentos, Cuando el sistema procesa las selecciones, Entonces no deberá ofrecer un campo "Monto a aplicar" editable por línea. La aplicación del monto del cobro a los documentos seleccionados sigue el orden en el que el usuario selecciona las proformas/facturas: el	Cubierto ▾	Backend NO ofrece campo "monto editable"; aplica en orden de selección (orden del array documentosS



	primer documento seleccionado se cubre primero, el segundo después, y así sucesivamente, hasta agotar el monto disponible del cobro. Si el cobro no alcanza para cubrir todos los documentos seleccionados, el último documento de la secuencia queda con saldo pendiente. El usuario controla el orden de aplicación mediante el orden de selección de los documentos.		eleccionados[])
D3	Dado que un cobro no cubre el adeudo total de un documento al que el usuario quiere aplicarlo, Cuando el usuario asocia el cobro al documento, Entonces el sistema deberá registrar la aplicación parcial. El documento queda con saldo pendiente y la asociación queda en espera del próximo cobro complementario que el usuario asociará en una sesión futura del Paso 2.	Cubierto ▾	Backend calcula aplicación parcial en Flujo 2; documento queda con saldo pendiente
D4	Dado que el usuario realizó una asociación, Cuando deselecciona uno o más cobros o documentos mediante checkbox, Entonces el sistema deberá permitir la deselección y recalculación dinámicamente el panel de Totales. El usuario puede reasociar múltiples veces mientras esté en el Paso 2.	Cubierto ▾	Frontend permite deseleccionar; backend recalcula dinámicamente en POST /calcular-saldo
D5	Dado que el usuario está en el Paso 2, Cuando intenta avanzar al Paso 3 mediante "Continuar",	Cubierto ▾	Backend valida en Flujo 3 Paso A1; retorna 400 si falta cobro o documento



	Entonces el sistema deberá validar que al menos un cobro y al menos un documento estén seleccionados. Si alguno de los dos está vacío, deberá mostrar un mensaje de error: "Debe seleccionar al menos un cobro y un documento para continuar".		
E1	Dado que el cliente tiene una o más Notas de Crédito vigentes (emitidas previamente con UUID timbrado SAT), Cuando el usuario asocia una proforma o factura, Entonces el sistema deberá mostrar el catálogo de NCs vigentes del cliente y permitir al usuario seleccionar libremente cuáles aplica al documento. La aplicación de NCs es OPCIONAL: el usuario puede aplicar cero, una o varias NCs según decida. Las NCs disponibles se muestran con: folio NC, monto disponible, fecha de emisión. Las NCs no seleccionadas siguen vigentes y disponibles para sesiones futuras del módulo Validar Cobro.	Pendiente ▾	GET /inicializar retornará NCs vigentes una vez R032 defina tabla y contrato; POST /calcular-saldo acepta ncsAplicadas: {idDoc: [idNCs]} opcionales — estructura del payload ya definida
E2	Dado que el usuario aplica una NC a una proforma o factura, Cuando el sistema procesa la aplicación, Entonces la NC se aplica por completo al documento seleccionado. Las NCs son por documento; no se distribuyen entre múltiples documentos en la misma sesión del Paso 2 (cada NC se aplica a un solo documento).	Cubierto ▾	Backend estructura ncsAplicadas como dict (idDoc → [idNCs]); implementación completa coordinada con R032
E3	Dado que el usuario quiere aplicar varias NCs a un mismo documento,	Cubierto ▾	Backend acepta array de NCs por documento;



	Cuando agrega NCs al documento, Entonces el sistema deberá permitir múltiples NCs aplicadas a la misma proforma o factura. La sumatoria de los montos de NCs aplicadas al documento se descuenta del adeudo.		suma todos los montos para restar del adeudo; implementación completa coordinada con R032
E4	Dado que el usuario aplica una NC al documento, Cuando el sistema registra la asociación, Entonces deberá preservar la referencia al UUID timbrado de la NC para que en el Paso 3, al emitir el CFDI correspondiente (factura, complemento de pago), las NCs se relacionen vía el nodo CFDIRelacionados con su UUID y monto correspondiente, conforme normativa SAT.	Cubierto ▾	El nombre del campo identificador de NC (UUID u otro) se define en R032; el contrato de este endpoint se actualizará al conocerlo
E5	Dado que el valor de una NC aplicada excede el adeudo del documento cubierto, Cuando el sistema procesa la aplicación, Entonces el saldo remanente de la NC no se gestiona como entidad re-aplicable. R16 no contempla auxiliar de saldos remanentes de NCs ni mecanismo para re-aplicar el remanente a una compra futura del cliente. Pendiente definir el tratamiento operativo del saldo remanente (auxiliar interno no fiscal, descarte operativo, otro).	Cubierto ▾	Backend NO gestiona saldo remanente; si $NC > adeudo$, solo aplica lo necesario; remanente se documenta como pendiente PROQUIFA
E6	Dado que el usuario finaliza el Paso 2 sin aplicar todas las NCs vigentes del cliente, Cuando el sistema procesa la asociación,	Pendiente ▾	Criterio de "vigencia" (columna, valor) se define en



	Entonces las NCs no seleccionadas en la sesión actual deberán mantenerse vigentes para el cliente, disponibles en sesiones futuras del módulo Validar Cobro. El sistema no descarta ni cierra automáticamente NCs no aplicadas. Pendiente verificar las reglas de vigencia de las NCs (caducidad, criterio de "vigente").		R032; la intención es no modificar NC si no fue seleccionada en la sesión
E7	Dado que el cliente NO tiene Notas de Crédito vigentes, Cuando el usuario selecciona un documento, Entonces el sistema deberá mostrar un mensaje indicativo: "No hay Notas de Crédito vigentes para este cliente" y permitir continuar con la asociación sin ofrecimiento de NCs.	Cubierto ▾	GET /pendingDocument retorna array vacío si no hay NCs; Frontend muestra mensaje indicativo
F0	Dado que el usuario opera el Paso 2, Cuando el sistema muestra el panel de Totales, Entonces deberá mostrar al pie de la pantalla, alineado a la derecha, los siguientes conceptos en este orden: "Adeudo Proformas y Facturas": suma de los importes totales de los documentos seleccionados (proformas + facturas), en positivo, en la moneda del cobro (con conversión por TC del Paso 1 cuando aplique). "NCs aplicadas": suma de los montos de las Notas de Crédito aplicadas a los documentos seleccionados, en negativo (con signo "-"). No se muestra si no hay NCs aplicadas.	Cubierto ▾	POST /validate-collection/client/{idCliente}/balance/calculate retorna: Adeudo, NCs aplicadas, Cobros aplicados, Adeudo restante, Saldo a favor (si aplica)



	• "Cobros aplicados": suma del monto aplicado de los cobros seleccionados a los documentos (no el monto capturado completo del cobro), en negativo (con signo "-"). • Línea de separación visual. • "Adeudo restante": resultado de Adeudo Proformas y Facturas – NCs aplicadas – Cobros aplicados. Debe ser \$0.00 en el escenario pago exacto. • "Saldo a favor generado": SOLO se muestra como línea adicional cuando los cobros disponibles exceden lo aplicado a documentos. Es la diferencia entre el monto capturado del cobro y el monto aplicado a documentos. • Todos los montos del panel se muestran en la moneda del cobro. El panel se actualiza en tiempo real con cada cambio de selección de cobros, NCs o documentos.		
F0.bis	Dado que el usuario selecciona uno o más documentos con moneda distinta a la moneda del cobro, Cuando el sistema muestra el panel de Totales y el listado de documentos, Entonces deberá convertir cada importe a moneda del cobro usando el TC capturado en el Paso 1 del cobro seleccionado, aplicado en el sentido necesario según el caso: • Cobro en moneda extranjera, documento en MXN: convertir el importe del documento (MXN) a moneda del cobro dividiéndolo entre el TC capturado. • Cobro en MXN, documento en moneda extranjera: convertir el importe del documento a MXN multiplicándolo por el	Cubierto ▾	Flujo 2 convierte importes con TC de Paso 1; retorna detalleConversiones[] con fórmula por documento



	TC capturado (que es el TC de la moneda del documento vs MXN, capturado anticipadamente porque el cliente factura en esa moneda). Cobro en moneda extranjera A, documento en moneda extranjera B distinta: escenario poco común, pendiente definir si aplica para PROQUIFA o requiere pivote MXN con dos TCs separados. En todos los casos, el sistema deberá mostrar en el listado de documentos el importe original con su moneda y el importe convertido a moneda del cobro. El TC aplicado y la fórmula de conversión se exponen vía tooltip (hover o click) sobre el importe convertido. Si la moneda del documento coincide con la moneda del cobro, no aplica conversión.		
F1	Dado que el usuario actualiza la asociación (selecciona cobros, NCs, documentos), Cuando el sistema recalcula, Entonces deberá calcular y mostrar dinámicamente en el panel de Totales: Adeudo Proformas y Facturas (suma de importes de documentos seleccionados convertidos a moneda del cobro vía TC capturado en el Paso 1 cuando aplique); NCs aplicadas (suma de NCs aplicadas a los documentos, convertidas a moneda del cobro cuando aplique, en negativo); Cobros aplicados (suma del monto aplicado de cobros a los documentos, no el capturado completo, en negativo); y Adeudo restante (Adeudo Proformas y Facturas – NCs aplicadas – Cobros aplicados). La actualización es en tiempo real ante cualquier cambio.	Cubierto ▾	POST /validate-collection/client/{idCliente}/balance/calculate recalcula en tiempo real ante cada cambio



F2	Dado que el Adeudo restante es exactamente \$0.00 y no hay excedente del cobro, Cuando el sistema lo detecta, Entonces deberá indicar visualmente "Pago exacto" en el panel de Totales y habilitar el botón "Continuar" sin condiciones adicionales. La línea "Saldo a favor generado" no se muestra en este escenario.	Cubierto ▾	Backend detecta saldo = \$0.00; retorna escenario="PAGO_EXACTO", puedeContinuar=true
F3	Dado que los cobros seleccionados exceden el monto necesario para cubrir el Adeudo Proformas y Facturas (descontadas las NCs aplicadas), Cuando el sistema lo detecta, Entonces deberá: cerrar el Adeudo restante en \$0.00 (los cobros se aplican parcialmente hasta cubrir el adeudo); mostrar la línea adicional "Saldo a favor generado" en el panel de Totales con el monto excedente; registrar el excedente como saldo a favor del cliente para reflejarse en el Estado de Cuenta/Auxiliar; marcar el cobro origen del excedente con identificador visible "Saldo a favor" en el listado del Paso 1 (para uso en sesiones futuras del módulo); y habilitar el botón "Continuar".	Cubierto ▾	Backend detecta saldo > 0; retorna escenario="SOBREPAGO", puedeContinuar=true; Flujo 3 INSERT fccSaldoFavor Cliente
F4	Dado que el Adeudo restante es positivo y su valor es ≤ 100 MXN (inclusiva del límite), Cuando el sistema evalúa, Entonces deberá: mostrar el Adeudo restante con etiqueta "Diferencia dentro de tolerancia" en color amarillo o equivalente; permitir cerrar el documento con el monto recibido mediante registro manual del operador; reflejar la diferencia como saldo pendiente en el Estado	Cubierto ▾	Backend detecta $0 < \text{saldo} \leq \text{umbral}$; retorna escenario="TOLERANCIA" (region-neutral), puedeContinuar=true; umbral leído



	de Cuenta/Auxiliar del cliente; y habilitar el botón "Continuar". La tolerancia 100 MXN es una Política Interna PROQUIFA. Cuando la moneda del cobro es distinta a MXN, pendiente confirmar si la tolerancia se convierte al equivalente del día ($100 \text{ MXN} \times \text{TC inverso}$), se mantiene en valor absoluto MXN, o aplica una tolerancia distinta.		de config Finanzas:ToleranciaValidarCobroMXN; Flujo 3 INSERT fccSaldoFavor Cliente con TipoSaldo="ToleranciaAplicada", ClaveRegion="MEX", ClaveMoneda="MXN"
F5	Dado que el Adeudo restante es positivo y su valor supera los 100 MXN, Cuando el sistema evalúa, Entonces deberá: mostrar el Adeudo restante con etiqueta "Diferencia excede tolerancia" en color rojo o equivalente; bloquear el botón "Continuar"; y sugerir al usuario marcar inconsistencia ("Pago Insuficiente" o "Pago Incompleto Vencido" según el caso) o dejar la asociación pendiente del próximo cobro complementario.	Cubierto ▾	Backend detecta saldo < -umbral; retorna escenario="PAGO_INSUFICIENTE", puedeContinuar=false; Flujo 3 retorna 409 Conflict
F6	Dado que el sistema convierte importes de documentos a la moneda del cobro, Cuando la conversión resulta en decimales que exceden dos posiciones (ej. \$123.456789), Entonces deberá redondear al alza a dos decimales (banquero's rounding o redondeo convencional según política PROQUIFA) y mostrar el importe redondeado en el panel de Totales y en el listado de documentos. El TC exacto y la fórmula de conversión se	Cubierto ▾	Backend aplica redondeo a 2 decimales; expone TC exacto + fórmula vía detalleConversiones[] en tooltip



	expondrán vía tooltip para que el usuario pueda verificar la precisión del cálculo.		
F7	Dado que el usuario no ha seleccionado ningún cobro, Cuando accede a la sección de documentos, Entonces el sistema deberá mostrar en el panel de Totales: "Cobros aplicados: \$0.00" y "Adeudo restante" igual al adeudo completo de los documentos seleccionados (sin reducciones). El botón "Continuar" estará deshabilitado con el mensaje: "Debe seleccionar al menos un cobro para continuar".	Cubierto ▾	Frontend valida selección antes de invocar cálculo; backend retorna error 400 si array está vacío
G1	Dado que el usuario detecta una inconsistencia, Cuando presiona "Marcar Inconsistencia en Cobro" en el pie de la pantalla, Entonces el sistema deberá abrir el modal "Inconsistencia de Pago".	Pendiente ▾	Modal de inconsistencia contemplado en R16A-RE-FU-024 — endpoint y lógica pendientes
G2	Dado que el modal está abierto, Cuando el sistema muestra los campos, Entonces deberá ofrecer: Tipo de Inconsistencia (obligatorio, combo del catálogo de tipos del Paso 2 que incluye los del Paso 1 más tipos contextuales) y Comentario adicional (opcional, texto libre para describir detalle adicional al cliente). Catálogo completo pendiente de definición por PROQUIFA. Tipos confirmados: "Pago Incompleto Vencido", "Pago Insuficiente".	Pendiente ▾	Modal de inconsistencia contemplado en R16A-RE-FU-024 — endpoint y lógica pendientes



G3	Dado que el modal está abierto, Cuando el usuario finaliza la captura, Entonces deberá ofrecer dos acciones: Cancelar (cierra el modal sin guardar) y Confirmar Inconsistencia (registra la inconsistencia).	Pendiente ▾	Modal de inconsistencia contemplado en R16A-RE-FU-024 — endpoint y lógica pendientes
G4	Dado que el usuario seleccionó "Pago Incompleto Vencido" como tipo de inconsistencia, Cuando confirma la inconsistencia, Entonces el sistema deberá habilitar adicionalmente la opción de marcar el pedido asociado al cobro con el estado "Pendiente de cancelación por falta de pago" o similar. Pendiente confirmar si este marcado debe ejecutar la cancelación fiscal efectiva de la factura asociada: el cliente definió en R16A-RE-FU-023 (Regla 9) que la cancelación manual con factura emitida sí detona cancelación fiscal ante el SAT; falta alinear si este escenario de inconsistencia sigue el mismo comportamiento o se mantiene como trabajo externo. Este marcado no dispara devolución del pago parcial recibido (no hay sistema de devoluciones en R16). El marcado únicamente notifica al área de Finanzas para gestión externa. Pendiente definir el comportamiento exacto del modal y el mecanismo de transferencia a Finanzas.	Pendiente ▾	Modal de inconsistencia contemplado en R16A-RE-FU-024 — endpoint y lógica pendientes
G5	Dado que el usuario seleccionó "Pago Insuficiente" como tipo de inconsistencia, Cuando confirma la inconsistencia,	Pendiente ▾	Modal de inconsistencia contemplado en R16A-RE-FU-024 — endpoint y



	Entonces el sistema deberá registrar la inconsistencia y dejar la asociación pendiente del próximo cobro complementario que el cliente realice. No se marca el pedido para cancelación automáticamente. La notificación al cliente sobre la inconsistencia detectada se gestiona posteriormente en el flujo. Pendiente confirmar el cuerpo del correo con el cliente.		lógica pendientes
G6	Dado que el usuario confirma una inconsistencia, Cuando el sistema procesa la confirmación, Entonces deberá mostrar un mensaje de éxito: "Inconsistencia registrada. La asociación queda pendiente de resolución" y cerrar automáticamente el modal. El usuario regresa al Paso 2 con la asociación preservada y el estado de la inconsistencia registrado.	Pendiente ▾	Modal de inconsistencia contemplado en R16A-RE-FU-024 — endpoint y lógica pendientes
G7	Dado que el usuario ya marcó una inconsistencia, Cuando intenta marcar otra inconsistencia en la misma sesión del Paso 2, Entonces el sistema deberá permitir la reedición: abrirá el modal con el tipo y comentario anterior, permitiendo al usuario modificar los valores. Solo la última versión confirmada se preservará; las versiones previas se descartan.	Pendiente ▾	Modal de inconsistencia contemplado en R16A-RE-FU-024 — endpoint y lógica pendientes
H1	Dado que el usuario opera en el Paso 2, Cuando modifica asociaciones, aplica NCs, marca inconsistencias o navega, Entonces el sistema deberá auto-guardar el estado del Paso 2. No existe botón "Guardar"	Cubierto ▾	Flujo 5 — POST /validate-collection/client/{idCliente}/association/draft upsert en



	manual. Al regresar al Paso 2 del mismo cliente tras una interrupción, el sistema deberá reanudar en el punto donde el usuario se quedó con el estado preservado, sin reiniciar el flujo.		fccEstadoPas o2EnProgreso ; reanuda al regresar tras interrupción
H2	Dado que el usuario realizó asociaciones en el Paso 2, Cuando regresa al Paso 2 o continúa trabajando, Entonces el sistema deberá permitir deshacer, reasociar, aplicar o remover NCs, y marcar inconsistencias adicionales. Las asociaciones son editables mientras el usuario esté en el Paso 2.	Cubierto ▾	Flujo 5 preserva estado editable; Flujo 3 fija asociaciones al avanzar a Paso 3
H3	Dado que el usuario opera el Paso 2, Cuando presiona "Regresar", Entonces el sistema deberá retornar al Paso 1 (Captura del Cobro). El estado del Paso 2 queda auto-guardado.	Cubierto ▾	Flujo 6 — Frontend invoca POST /validate-collec tion/client/{idCl iente}/associat ion/draft; backend preserva estado; navega a Paso 1
H4	Dado que la asociación está en un escenario válido para avanzar (pago exacto, sobrepago con saldo a favor, o pago de menos dentro de tolerancia ≤ 100 MXN), Cuando el usuario presiona "Continuar", Entonces el sistema deberá avanzar al Paso 3 (Facturación y Envío) con la asociación cerrada. A partir de este momento, las	Cubierto ▾	Flujo 3 valida escenario; permite avance solo si exacto, sobrepago, o tolerancia ≤ 100 MXN



	asociaciones quedan fijas para la emisión de documentos fiscales.		
H5	Dado que el escenario es pago de menos con diferencia > 100 MXN sin inconsistencia marcada, Cuando el usuario presiona "Continuar", Entonces el sistema deberá bloquear el avance y mostrar al usuario que debe completar la asociación, marcar inconsistencia, o dejar la asociación pendiente.	Cubierto ▾	Flujo 3 retorna 409 Conflict si escenario = PAGO_INSUFICIENTE
H6	Dado que el usuario realizó cambios en el Paso 2 (asociaciones, NCs, inconsistencias) sin avanzar al Paso 3, Cuando intenta abandonar el Paso 2 sin haber presionado "Continuar" (cierre del navegador, timeout de sesión, etc.), Entonces el sistema deberá auto-guardar el estado y permitir que el usuario regrese al Paso 2 en una sesión futura con los cambios preservados. No hay pérdida de datos.	Cubierto ▾	Flujo 5 auto-guarda cada 10s; cierre de navegador no causa pérdida (estado recuperable)
H7	Dado que el sistema auto-guardó el estado del Paso 2, Cuando ha transcurrido un período de inactividad (sesión expirada), Entonces el sistema deberá preservar el estado auto-guardado por al menos 30 días calendario (o según política de PROQUIFA). Al regresar el usuario, el estado se recupera automáticamente. Pendiente confirmar la política de expiración con IT/PROQUIFA.	Pendiente ▾	Política PROQUIFA de expiración >30 días pendiente confirmación





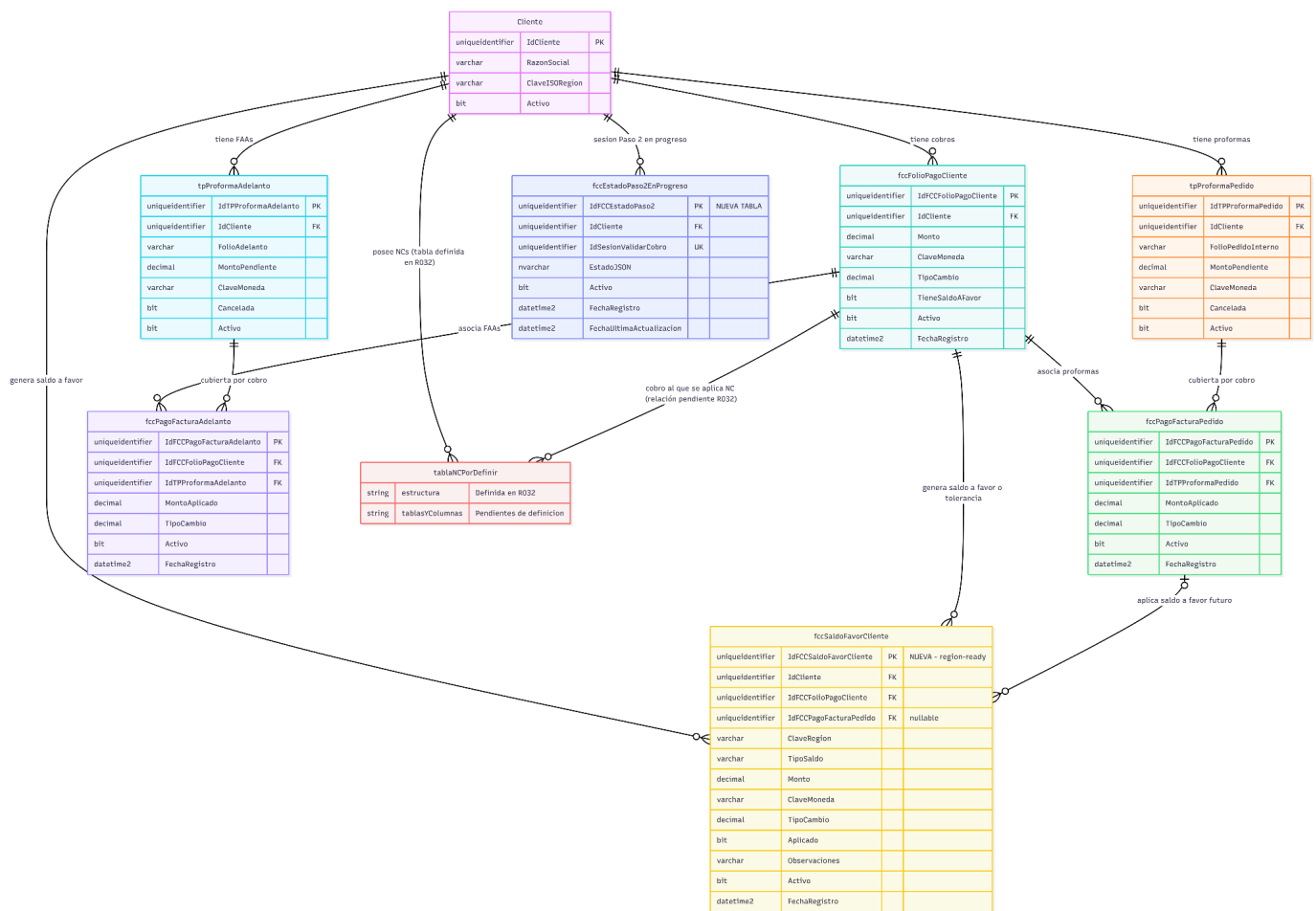
8. Reglas técnicas aplicadas

Regla	Descripción
RT-01	Cálculo saldo debe usar TC capturado en Paso 1, nunca consultar TC en tiempo real durante Paso 2. Cuando el usuario selecciona N cobros con TCs distintos, se usa el TC del primer cobro seleccionado como TC unificador de la sesión; los demás cobros se convierten a la moneda del primer cobro usando ese mismo TC
RT-02	Toda persistencia de asociaciones es transaccional ACID. Si falla cualquier INSERT/UPDATE, rollback completo
RT-03	Orden de selección de documentos determina aplicación de monto cobro. Primer documento se cubre primero
RT-04	Tolerancia bifurca flujo por umbral configurable: $\leq$ umbral permite avance + registra tolerancia. $>$ umbral bloquea. Para MX: umbral = Finanzas:ToleranciaValidarCobroMXN (100 por defecto). Nunca hardcodear 100 en lógica.
RT-05	NCs son opcionales y por documento. NC aplicada a doc A no puede aplicarse a doc B en misma sesión
RT-06	Datos de frontend siempre validados. Cobros/docs/NCs deben existir, montos deben ser positivos, TC debe ser > 0
RT-07	Todos los DTOs retornados incluyen moneda (MXN, USD, etc.) y TC aplicado en conversiones
RT-08	SaldoFavorCliente registra TipoSaldo ('SaldoFavor' vs 'ToleranciaAplicada') y flag Aplicado para trazabilidad
RT-09	Estado Paso 2 en progreso (fccEstadoPaso2EnProgreso) es JSON y se serializa/deserializa sin pérdida de información
RT-10	Una vez usuario avanza a Paso 3, asociaciones están FIJAS. No se permite edición desde Paso 2 posterior
RT-11	Toda lógica region-específica vive en la strategy (Paso2MxStrategy), no en el controller ni en el service base. R026 entrega la interfaz + base + implementación MX. RE-FU-027 agrega su strategy sin tocar nada de lo entregado
RT-12	La detección de región usa VCliente.ClaveISORegion (string enum síncrono). Nunca Cliente.IdRegion (async, puede ser undefined mientras

	carga catálogo). La región del cliente (wizard) es distinta a la del usuario logueado.
RT-13	fccSaldoFavorCliente modela moneda con ClaveMoneda varchar(3), no con columnas bit por moneda (MXN, USD, PEN). Esto evita ALTER TABLE por cada moneda nueva. Coherente con el resto del modelo.

Diseño de componentes

1. Diagramas



2. Servicios REST

GET /api/v2/validar-cobro/paso-2/inicializar

Responsabilidad: Carga inicial del Paso 2 con documentos pendientes, NCs vigentes y estado previo. Los cobros ya vienen del Paso 1 (NgRx state) — no se re-consultan aquí.

Request:

```
GET /api/v1/validate-collection/client/{idCliente}/association/draft
Headers:
  Authorization: Bearer {token}
  X-IdSesionValidarCobro: {uuid}
```

Response 200 OK:

```
{
  "idCliente": "uuid",
  "documentosPendientes": [ ... ],
  "notasCreditoVigentes": [ ... ],
  "estadoEnProgreso": { ... }
}
```

Response 400/500:

```
{
  "type": "https://tools.ietf.org/html/rfc7231#section-6.5.1",
  "title": "Client not found or no payments captured",
  "status": 400,
  "detail": "El cliente no existe, o no tiene cobros capturados en el Paso 1.",
  "instance":
"/api/v1/validate-collection/client/{idCliente}/pendingDocument",
  "code": "ERR_INVALID_CLIENT"
}
```

POST /api/v1/validate-collection/client/{idCliente}/balance/calculate

Responsabilidad: Cálculo dinámico de saldo multi-divisa. Invocado cada vez que usuario cambia selección.

Request:

```
{
```



```
{
  "clientId": "uuid",
  "selectedPayments": ["uuid-payment-1"],
  "selectedDocuments": ["uuid-doc-1"],
  "appliedCreditNotes": {"uuid-doc-1": ["uuid-creditnote-1"]}
}
```

Response 200 OK:

```
{
  "paymentSum": 5000.00,
  "creditNoteSum": 500.00,
  "debtSum": 4500.00,
  "associationBalance": 1000.00,
  "scenario": "SALDO_A_FAVOR",
  "canContinue": true,
  "finalCurrency": "MXN",
  "appliedToleranceThreshold": 100.00,
  "conversionDetails": [
    {
      "documentId": "uuid-doc-1",
      "originalCurrency": "MXN",
      "originalAmount": 4500.00,
      "exchangeRate": 1.00,
      "convertedAmount": 4500.00,
      "formula": "4500.00 × 1.00 = 4500.00"
    }
  ]
}
```

POST /api/v2/validar-cobro/paso-2/confirmar-asociacion

Responsabilidad: Persistencia transaccional de asociaciones. Ejecuta Flujo 3.

Request:

```
{
  "clientId": "uuid",
  "idSessionId": "uuid",
  "selectedPayments": ["uuid-payment-1"],
  "selectedDocuments": ["uuid-doc-1"],
}
```




```
"appliedCreditNotes": {"uuid-doc-1": ["uuid-creditnote-1"]},
"scenario": "SALDO_A_FAVOR",
"associationBalance": 1000.00
}
```

Response 200 OK:

```
{
  "success": true,
  "associationId": "uuid-session",
  "message": "Association confirmed.",
  "scenario": "SALDO_A_FAVOR",
  "registeredBalance": 1000.00,
  "fiscalEffect": true,
  "sunatNote": null
}
```

Response 409 Conflict (escenario insuficiente > 100MXN sin inconsistencia):

```
{
  "type": "https://tools.ietf.org/html/rfc7231#section-6.5.8",
  "title": "Insufficient balance",
  "status": 409,
  "detail": "Insufficient balance. Flag inconsistency or leave pending.",
  "instance":
"/api/v1/validate-collection/client/{idCliente}/associationConfirmation",
  "code": "ERR_INSUFFICIENT_BALANCE",
  "insufficientBalance": 500.00
}
```

POST /api/v2/validar-cobro/paso-2/guardar-progreso

Responsabilidad: Auto-guardado del estado Paso 2 en progreso.

Request:

```
{
```



```
"clientId": "uuid",
"idSessionId": "uuid",
"selectedPayments": ["uuid-payment-1"],
"selectedDocuments": ["uuid-doc-1"],
"appliedCreditNotes": {"uuid-doc-1": ["uuid-creditnote-1"]},
"inconsistency": null,
"timestamp": 1719701400
}
```

Response 200 OK:

```
{
  "success": true,
  "message": "Progress saved automatically."
}
```



Impacto Técnico

1. Impacto en código existente

Alteraciones:

Tabla	Cambio	Impacto
fccNotaCredito	ADD FK IdFCCPagoClient e	Trazabilidad. No impacta consultas existentes (FK nullable)
catTipoInconsistenciaCobro	(pendiente — scope R16A-RE-FU-024)	Flag AplicaMarkPendienteCancelacion y demás alteraciones relacionadas al modal de inconsistencia se definen en el diseño de R16A-RE-FU-024
tpPedido	(pendiente — scope R16A-RE-FU-024)	Campo para marcar pedido pendiente de cancelación se define en R16A-RE-FU-024

Reutilización:

- fccPagoFacturaPedido — Ya existe, se usan para asociar cobro ↔ proforma
- fccPagoFacturaAdelanto — Ya existe, se usan para asociar cobro ↔ FAA
- fccNotaCredito — Ya existe, se marcan Aplicada=1 si se aplican
- fccPagoCliente — Ya existe, se leen TipoCambio e Aplicada para validaciones

2. Impacto en modelos por actualización de modelos

Entidad	Tipo de cambio	Detalle
fccSaldoFavorCliente	Nueva tabla	Registra sobrepago + tolerancia. Usa ClaveMoneda varchar(3) + ClaveRegion varchar(3) en lugar de MXN/USD bit. R027 inserta sin ALTER



fccEstadoPaso2EnProgreso	Nueva tabla	Almacena estado JSON del Paso 2 en progreso. Ver DDL T4
Tabla NC (nombre por definir)	Pendiente R032	La tabla se crea en R032. Una vez definida, se coordinará con ese desarrollador para agregar FK hacia fccPagoCliente y registrar a qué cobro se aplicó cada NC
catTipoInconsistenciaCobro	Pendiente — scope R16A-RE-FU-024	Alteraciones relacionadas al modal de inconsistencia se especifican en R16A-RE-FU-024 (diseño no realizado)
tpPedido	Pendiente — scope R16A-RE-FU-024	Campo para marcar cancelación se especifica en R16A-RE-FU-024
fccPagoFacturaPedido	Reutilización	Sin cambios de esquema. Se insertan registros nuevos en Flujo 3
fccPagoFacturaAdelanto	Reutilización	Sin cambios de esquema. Se insertan registros nuevos en Flujo 3
fccPagoCliente	Reutilización	Sin cambios de esquema. Se actualiza TieneSaldoAFavor en Flujo 3

Endpoint expuestos

solución/Proyecto	Endpoint	Parametros	Salida
ProquifaDotNet.Finanzas	GET /api/v1/validate-collection/client/{idCliente}/pendingDocument	idCliente (UUID, query); X-IdSesionValidarCobro (UUID, header)	region, monedaBase, umbralTolerancia, empresasEmisoras[], documentosPendientes[], notasCreditoVigentes[] (estructura pendiente)



			R032), estadoEnProgreso
ProquifaDotNet.Finanzas	POST /api/v1/validate-collection/client/{idCliente}/balance/calculate	idCliente (UUID, query); X-IdSesionValidarCobro (UUID, header)	sumaCobros, sumaNCs, sumaAdeudo, saldoAsociacion, escenario (PAGO_EXACTO/SOBREPAGO/TOLERANCIA/PAGO_INSUFICIENTE), puedeContinuar, monedaFinal, umbralToleranciaAplicado, detalleConversiones[]
ProquifaDotNet.Finanzas	POST /api/v1/validate-collection/client/{idCliente}/associationConfirmation	idCliente, idSesionValidarCobro, cobrosSeleccionados[], documentosSeleccionados[], ncsAplicadas{}, escenario, saldoAsociacion	exito, idAsociacion, mensaje, escenario, saldoFavorRegistrado, efectoFiscal, notaSunat
ProquifaDotNet.Finanzas	POST /api/v1/validate-collection/client/{idCliente}/association/draft	idCliente, idSesionValidarCobro, cobrosSeleccionados[], documentosSeleccionados[], ncsAplicadas{}, inconsistencia, timestamp	exito, mensaje

Manejo de Errores y Excepciones

Escenario	Respuesta HTTP	Comportamiento
Cliente no existe o región ≠ México	400 Bad Request	detail: "Cliente inválido o no es región México", code: "ERR_INVALID_CLIENT"



Cobro no existe o ya fue aplicado	404 Not Found	detail: "Cobro no encontrado o ya está aplicado", code: "ERR_PAYMENT_NOT_FOUND"
Documento no existe o ya está cancelado	404 Not Found	detail: "Documento no encontrado o cancelado", code: "ERR_DOCUMENT_NOT_FOUND"
NC no existe o ya está aplicada	404 Not Found	detail: "Nota de Crédito no encontrada o ya aplicada", code: "ERR_CREDITNOTE_NOT_FOUND"
TC del cobro es NULL o ≤ 0	400 Bad Request	detail: "TC capturado en Paso 1 inválido", code: "ERR_INVALID_EXCHANGE_RATE"
Transacción falla en Paso A2-A8 (BD)	500 Internal Server Error	Rollback automático. detail: "Error al persistir asociación. Reintente.", code: "ERR_TRANSACTION_FAILED"
Timeout en llamada a ProquifaDotNet (>30s)	504 Gateway Timeout	detail: "Servicio no disponible. Reintente.", code: "ERR_UPSTREAM_TIMEOUT"
Usuario intenta continuar con saldo insuficiente >100 MXN sin inconsistencia	409 Conflict	detail: "Saldo insuficiente. Marque inconsistencia.", code: "ERR_INSUFFICIENT_BALANCE"
Sesión Paso 2 expirada (>24h)	401 Unauthorized	detail: "Sesión expirada. Reinicie el flujo.", code: "ERR_SESSION_EXPIRED"



Estrategia de Pruebas (Diseño de las pruebas)

1. Pruebas funcionales (Criterios de Aceptación en DEV)

- ☐ B1 — GET /inicializar retorna cobros con folio, monto, fecha y flag tieneSaldoAFavor
- ☐ C1 — NC con monto > adeudo: sistema aplica NC, no genera remanente re-aplicable, no rompe la transacción
- ☐ E6 — POST /calcular-saldo retorna sumaCobros, sumaNCs, sumaAdeudo, saldoAsociacion correctos ante cada cambio de selección
- ☐ F0 — Con cobro en MXN y doc en USD: $\text{importe convertido} = \text{importeUSD} \times \text{TC}$. Con cobro en USD y doc en MXN: $\text{importe convertido} = \text{importeMXN} \div \text{TC}$
- ☐ F1 — (pendiente R16A-RE-FU-024) — Modal de inconsistencia y registro en fccInconsistenciaCobro se verifican cuando ese requisito tenga diseño y endpoint definidos
- ☐ H1 — Auto-guardado: cerrar sesión y reabrir Paso 2 recupera cobros/docs/NCs seleccionados previamente
- ☐ H2 — POST /guardar-progreso preserva estado al regresar; estado recuperable al reingresar al Paso 2
- ☐ H4 — POST /confirmar-asociacion con saldo insuficiente >100 MXN retorna 409 con puedeContinuar=false

2. Pruebas técnicas (unitarias e integración)

Calcular Saldo

- ☐ Pago exacto: cobros = docs → saldo 0, PAGO_EXACTO, avanza ☒
- ☐ Sobre pago: cobros > docs → saldo positivo, SOBREPAGO, avanza ☒
- ☐ Tolerancia: cobros < docs pero $|\text{saldo}| \leq 100$ → TOLERANCIA_100_MXN, avanza ☒
- ☐ Insuficiente: $|\text{saldo}| > 100$ → PAGO_INSUFICIENTE, bloquea ☒
- ☐ Límite exacto: saldo = -100 (borde) → TOLERANCIA ☒
- ☐ Límite fallido: saldo = -100.01 → INSUFICIENTE ☒

Multi-divisa

- ☐ Cobro MXN, doc USD → divide importe ÷ TC ☒
- ☐ Cobro USD, doc MXN → multiplica importe × TC ☒
- ☐ TC = 1.00 (pase identidad) ☒
- ☐ TC = 17.5 (México realista) ☒
- ☐ Redondeo: 4 decimales máximo ☒
- ☐ Fórmula retorna string correcto ☒



Notas de crédito

- ☐ Sin NCs → suma NCs = 0 ✓
- ☐ 1 NC aplicada → suma correcta ✓
- ☐ Múltiples NCs en 1 doc → suma total ✓
- ☐ NC en distinta divisa que cobro → convierte correctamente ✓
- ☐ NC > adeudo → suma completa (no remanente) ✓

Precisión Decimal

- ☐ Usar decimal no double → no pierda centavos ✓
- ☐ $5000.01 + 2500.99 = 7501.00$ (no 7500.99999) ✓
- ☐ Division exacta: $4500 \div 1.17$ sin truncar ✓

Validación entrada

- ☐ Cobro NULL → error 400 ✓
- ☐ TC NULL o ≤ 0 → error 400 ✓
- ☐ Documento NO existe → error 404 ✓
- ☐ NC ya aplicada → error 400 ✓
- ☐ Array documentos vacío → error 400 ✓

DTOs respuesta

- ☐ POST /calcular-saldo retorna todos campos requeridos ✓
- ☐ detalleConversiones[] tiene estructura correcta ✓
- ☐ escenario tiene valor válido (enum) ✓
- ☐ puedeContinuar es booleano correcto ✓

Guardar progreso

- ☐ INSERT primer guardado ✓
- ☐ UPDATE reutiliza ID (UPSERT) ✓
- ☐ JSON serializa sin pérdida ✓
- ☐ Deserializa igual que entrada ✓
- ☐ Timestamp se preserva ✓

3. Casos críticos

- Red Inestable: Simular timeout en llamada a ProquifaDotNet (validar reintento)
- Tolerancia Bifurcada: Probar ≤ 100 vs > 100 MXN con precisión decimal (usar decimal no double)
- Multi-divisa: Diferentes combinaciones MXN↔USD↔EUR, verificar fórmulas conversión
- Transacción Rollback: Inyectar error en Paso A5 (INSERT fccSaldoFavorCliente), validar rollback
- Estado Expirado: Sesión Paso 2 >24h, validar limpieza automática



- Concurrencia: 2 usuarios simultáneamente en Paso 2 del mismo cliente, validar aislamiento (isolation level)



Control de versiones

Versión	Fecha	Autor	Tipo de Cambio	Descripción del cambio	Aprobó
1.0	06/03/25	Roberto Baez ...	Creación ▾	Se crea plantilla genérica	Rose Ríos ...
1.1	16/06/26	Alan Fernand...	Creación ▾	Se crea plantilla genérica para DIS	
1.2	01/07/26	Isai Amaury Gar...	Actualiz... ▾	Documentar diseño del requisito	

